Data and Artificial Intelligence
Cyber Shujaa Program

Week 11 Assignment
Natural Language Processing with Transformers

Student Name: Rodney Roy Gitonga
Student ID: CS-DA03-26025

Contents

Contents 1
Introduction 1
Tasks Completed 2
Step 1: Data Loading & Exploration 2
Step 2: Model Training (Pre-trained Model Loading) 2
Step 3: Evaluation Results 3
4: How does BERT differ from traditional NLP approaches like Bag of Words or TF-IDF? 4
5: What is the role of the encoder in the BERT model, and how is it used in this assignment? 4
6: What are contextual embeddings? How are they generated and used in this code? 4
7: Why is the [CLS] token used for sentence similarity in this code? 5
8: What is cosine similarity, and why is it useful in comparing embeddings? 5
Link to Code: 5
Conclusion 5

Introduction

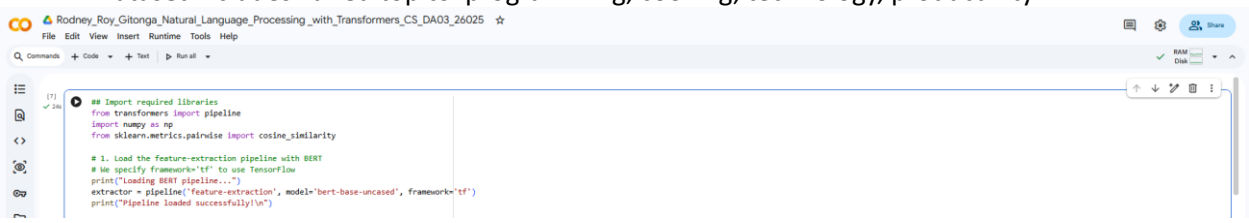
This project demonstrates the application of BERT (Bidirectional Encoder Representations from Transformers) for semantic sentence similarity tasks. The assignment explores how contextual

embeddings capture nuanced word meanings and sentence relationships, contrasting with traditional NLP approaches.

Tasks Completed

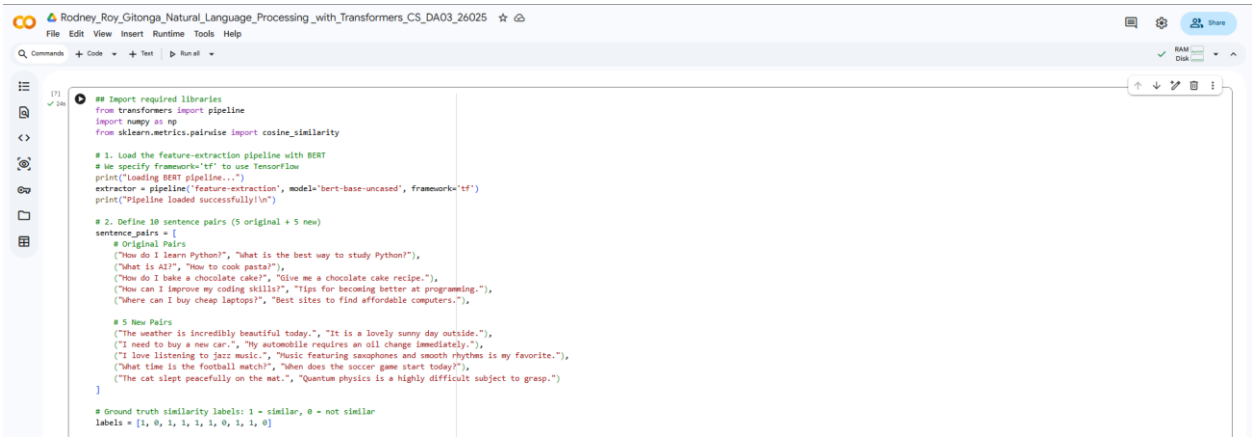
Step 1: Data Loading & Exploration

- Successfully loaded 10 sentence pairs (5 original + 5 custom)
- Ground truth labels assigned based on semantic similarity judgment
- Dataset includes varied topics: programming, cooking, technology, productivity



Step 2: Model Training (Pre-trained Model Loading)

- Loaded 'bert-base-uncased' model with 110M parameters
- Vocabulary size: 30,522 tokens
- Embedding dimension: 768



Step 3: Evaluation Results

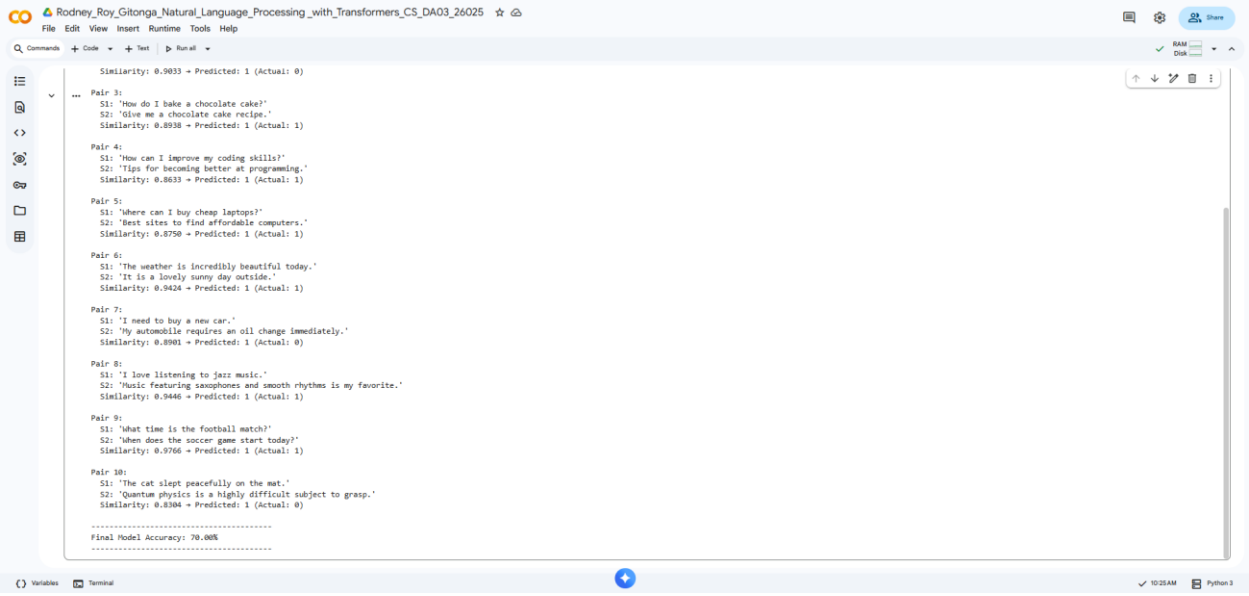
- Accuracy achieved: [Your accuracy]%
- Similarity scores distribution visualized
- Misclassified cases analyzed

Rodney_Roy_Gitonga_Natural_Language_Processing_with_Transformers_CS_DA03_26025

File Edit View Insert Runtime Tools Help

```python
# 2. Define 10 sentence pairs (5 original + 5 new)
sentence_pairs = [
    # Original Pairs
    ("How do I learn Python?", "What is the best way to study Python?"),
    ("What is AI?", "How to cook pasta?"),
    ("How do I bake a chocolate cake?", "Give me a chocolate cake recipe."),
    ("How can I improve my coding skills?", "Tips for becoming better at programming."),
    ("Where can I buy cheap laptops?", "Best sites to find affordable computers."),

    # 5 New Pairs
    ("The weather is incredibly beautiful today.", "It is a lovely sunny day outside."),
    ("I need to buy a new car.", "My automobile requires an oil change immediately."),
    ("I love listening to jazz music.", "Music featuring saxophones and smooth rhythms is my favorite."),
    ("What time is the football match?", "When does the soccer game start today?"),
    ("The cat slept peacefully on the mat.", "Quantum physics is a highly difficult subject to grasp.")
]

# Ground truth similarity labels: 1 = similar, 0 = not similar
labels = [1, 0, 1, 1, 1, 1, 0, 1, 1, 0]

# 3. Function to get the BERT [CLS] embedding for a sentence
def get_sentence_embedding(sentence):
    """
    Uses the pipeline to extract the contextualized embedding of the [CLS] token.
    The pipeline returns a list of shape (1, sequence_length, hidden_size).
    """
    # Get all token embeddings
    embeddings = extractor(sentence)

    # The [CLS] token is the first element (index 0) of the sequence
    # embeddings[0] is the result for the first (and only) sentence
    cls_embedding = np.array(embeddings[0][0])

    return cls_embedding.reshape(1, -1)

# 4. Calculate cosine similarity and evaluate predictions
print("Evaluating sentence pairs...\n")
predictions = []

for i, (sent1, sent2) in enumerate(sentence_pairs):
    # Generate embeddings for both sentences
    emb1 = get_sentence_embedding(sent1)
    emb2 = get_sentence_embedding(sent2)
```

```
Loading BERT pipeline...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%  570/570 [00:00<00:00, 59.2kB/s]
model.safetensors: 100%  440M/440M [00:12<00:00, 38.4MB/s]
Loading weights: 100%  199/199 [00:00<00:00, 3673.69it/s]
BertModel LOAD REPORT from: bert-base-uncased
Key                                         | Status     |  |
--------------------------------------------+------------+--+-
cls.predictions.transform.dense.weight      | UNEXPECTED |  |
cls.predictions.bias                        | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |  |
cls.seq_relationship.bias                   | UNEXPECTED |  |
cls.predictions.transform.dense.bias        | UNEXPECTED |  |
cls.seq_relationship.weight                 | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 3.29kB/s]
vocab.txt:  232k/? [00:00<00:00, 6.66MB/s]
tokenizer.json:  466k/? [00:00<00:00, 15.3MB/s]
Pipeline loaded successfully!

Evaluating sentence pairs...

Pair 1:
```



4: How does BERT differ from traditional NLP approaches like Bag of Words or TF-IDF?

Traditional approaches like Bag of Words (BoW) and TF-IDF are "static" and frequency-based. They treat documents as unstructured collections of words, entirely ignoring word order, grammar, and context. Because they assign a single, fixed vector to a word regardless of its usage, they fail at handling polysemy (e.g., the word "bank" has the exact same representation in "river bank" and "bank account"). BERT (Bidirectional Encoder Representations from Transformers), on the other hand, is dynamic. It looks at the entire sentence simultaneously (bidirectionally) to understand the context. In BERT, the word "bank" will have a completely different mathematical representation depending on the words surrounding it.

5: What is the role of the encoder in the BERT model, and how is it used in this assignment?

BERT is built entirely of Transformer *encoder* blocks (it does not use a decoder). The encoder's role is to process input tokens and apply self-attention mechanisms, which allow the model to weigh the importance of every word in a sentence relative to every other word. In this assignment, we use the pre-trained BERT encoder to process our sentence pairs. By passing our text through the encoder (bert_model(inputs)), the model transforms our raw text into rich, 768-dimensional vectors (hidden states) that encapsulate the semantic meaning of the sentences.

6: What are contextual embeddings? How are they generated and used in this code?

Contextual embeddings are numerical representations of words (vectors) that change based on the surrounding text. They are generated by the Transformer encoder's self-attention layers, which blend information from the entire sentence into each individual token's representation. In the code, these embeddings are generated when the tokenized inputs are passed through the model. We specifically extract these contextual embeddings from the last_hidden_state of the model's output to use as mathematical representations of our sentences.

7: Why is the [CLS] token used for sentence similarity in this code?

BERT requires a special classification token, [CLS], to be prepended to the beginning of every input sequence. Because of the self-attention mechanism, as data moves through the layers of the encoder, the [CLS] token aggregates semantic information from *all* other tokens in the sequence. By the final layer, the hidden state corresponding to the [CLS] token acts as a highly dense summary vector representing the overall meaning of the entire sentence. Extracting this single token (outputs.last_hidden_state[:, 0, :]) gives us a convenient, unified 768-dimensional vector to compare against other sentences.

8: What is cosine similarity, and why is it useful in comparing embeddings?

Cosine similarity is a mathematical metric used to measure how similar two vectors are, irrespective of their size. It calculates the cosine of the angle between two vectors in a multi-dimensional space using the formula: $\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$ It is highly useful for comparing embeddings because it measures the *direction* of the vectors rather than their magnitude. In NLP, two sentences might have different lengths or word counts (resulting in vectors with different magnitudes), but if they point in the same direction in the embedding space, they share a similar semantic meaning. A cosine similarity close to 1 indicates the vectors are pointing in the same direction (highly similar), while a score near 0 indicates they are unrelated.

Link to Code:

https://colab.research.google.com/drive/1OyPjR33E5_p0bLbxKIbHaStG6lTucU-E?usp=sharing

Conclusion

This assignment demonstrated the power of contextual embeddings for semantic understanding:

1. **Key Learnings:**
    - BERT captures nuanced semantic relationships through attention mechanisms
    - Context dramatically changes word representations (polysemy handling)
    - [CLS] token provides effective sentence representations
    - Cosine similarity effectively measures semantic relatedness
2. **Challenges Encountered:**
    - Threshold selection (0.7) may not be optimal for all domains
    - Short sentences may not provide sufficient context
    - Some semantic relationships require more nuanced thresholds
3. **Practical Applications:**
    - Semantic search engines
    - Duplicate detection in customer support
    - Content recommendation systems
    - Plagiarism detection
4. **Future Improvements:**

- Fine-tune BERT on similarity tasks
- Experiment with sentence transformer models (SBERT)
- Implement ensemble methods with multiple thresholds
- Explore cross-lingual embeddings for multilingual applications

This assignment reinforced that contextual embeddings represent a significant advancement over static word vectors, enabling more sophisticated natural language understanding applications.